

VALUE ADDED PROBLEM-1 (Experiment – 11)

/* Name-Simrat Singh Arora

/PRN-25070521181/

SECTION-D-I

Value Added Problem-1 */

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
struct Node {
```

```
    int data;
```

```
    struct Node* left;
```

```
    struct Node* right;
```

```
};
```

```
struct Node* createNode(int data) {
```

```
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
```

```
    newNode->data = data;
```

```
    newNode->left = newNode->right = NULL;
```

```
    return newNode;
```

```
}
```

```
struct Node* insert(struct Node* root, int data) {
```

```
    if (root == NULL)
```

```
        return createNode(data);
```

```
    if (data < root->data)
```

```
        root->left = insert(root->left, data);
```

```
    else
```

```
        root->right = insert(root->right, data);
```

```
    return root;
```

```
}
```

```
void printInRange(struct Node* root, int low, int high) {
```

```
    if (root == NULL)
        return;

    if (root->data > low)
        printInRange(root->left, low, high);

    if (root->data >= low && root->data <= high)
        printf("%d ", root->data);

    if (root->data < high)
        printInRange(root->right, low, high);
}

int main() {
    struct Node* root = NULL;

    root = insert(root, 17);
    insert(root, 4);
    insert(root, 18);
    insert(root, 2);
    insert(root, 9);
    int low = 4, high = 24;
    printf("Output: ");
    printInRange(root, low, high);

    return 0;
}
```

main.c

   Share

Run

Output

Clear

```
1- /* Name-Simrat Singh Arora
2- /PRN-25070521181/
3- SECTION-D-I
4- Value Added Problem-1 */
5- #include <stdio.h>
6- #include <stdlib.h>
7- struct Node {
8-     int data;
9-     struct Node* left;
10-    struct Node* right;
11- };
12- struct Node* createNode(int data) {
13-     struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
14-     newNode->data = data;
15-     newNode->left = newNode->right = NULL;
16-     return newNode;
17- }
18- struct Node* insert(struct Node* root, int data) {
19-     if (root == NULL)
20-         return createNode(data);
21-     if (data < root->data)
22-         root->left = insert(root->left, data);
23-     else
24-         root->right = insert(root->right, data);
25-     return root;
26- }
27- void printInRange(struct Node* root, int low, int high) {
28-     if (root == NULL)
29-         return;
```

Output: 4 9 17 18

=== Code Execution Successful ===